

Agent-Mediated Knowledge Graph Chat — Low Level Design (LLD)

Scope: This LLD defines the database schema and exact query examples for Neo4j (Cypher) and ArangoDB (AQL), plus additional backend contracts needed to implement the MVP. It focuses on node/edge storage, versioning, staleness propagation, and merge requests.

1. Graph Model (concepts)

Node Label(s) / Collections

- `Node` (Neo4j label) / `nodes` (Arango collection)
- `id` (uuid) — primary key
- `name` (string)
- `type` (string: `main|component|child|continuation`)
- `status` (string: `fresh|stale|merge_pending|archived`)
- `vector_index_id` (string) — pointer to vector index
- `metadata` (json)
- `created_at`, `updated_at` (timestamps)

Edge Types / Edge Collections

- `DEPENDS_ON` / `depends` — parent depends on target
- `RELATED_TO` / `related` — loose relation
- `CONFLICTS_WITH` / `conflicts` — semantic conflict
- `HAS_VERSION` / `versions` — points from node -> version

Version (commit) objects

- `Version` (label / collection)
- `commit_id` (sha-like string)
- `node_id` (uuid)
- `content_snapshot` (blob or pointer to file storage)
- `diff_summary` (text)
- `created_at`, `author`

MergeRequest

- `merge_id` (uuid)
- `child_id`, `parent_id` (uuids)
- `status` (`pending|approved|rejected`)
- `text_diff` (unified diff or JSON Patch)
- `diff_summary` (NL)

- created_at, requested_by

HallucinationLog (embedded or separate collection)

- id, node_id, type, snippet, created_at

2. Neo4j (Cypher) — Schema setup & sample queries

Indexes & Constraints

```
CREATE CONSTRAINT node_id_unique IF NOT EXISTS
FOR (n:Node) REQUIRE n.id IS UNIQUE;

CREATE INDEX node_name_index IF NOT EXISTS FOR (n:Node) ON (n.name);
CREATE INDEX node_status_index IF NOT EXISTS FOR (n:Node) ON (n.status);

CREATE CONSTRAINT version_commit_unique IF NOT EXISTS
FOR (v:Version) REQUIRE v.commit_id IS UNIQUE;
```

Create Node

```
CREATE (n:Node {
  id: $id,
  name: $name,
  type: $type,
  status: 'fresh',
  vector_index_id: $vector_index_id,
  metadata: $metadata,
  created_at: datetime(),
  updated_at: datetime()
})
RETURN n;
```

Create Dependency Edge

```
MATCH (a:Node {id:$from}), (b:Node {id:$to})
MERGE (a)-[r:DEPENDS_ON]->(b)
ON CREATE SET r.created_at = datetime()
RETURN r;
```

Fetch dependencies for Router (one-level + transitive up to depth N)

```
MATCH (n:Node {id:$node_id})-[:DEPENDS_ON*0..3]->(dep)
RETURN dep.id AS id, dep.name AS name, dep.status AS status,
dep.vector_index_id AS vid
ORDER BY dep.updated_at DESC;
```

Mark dependents as stale (propagate)

```
MATCH (updated:Node {id:$updated_id})
MATCH (dependent)-[:DEPENDS_ON]->(updated)
SET dependent.status = 'stale', dependent.updated_at = datetime()
RETURN dependent.id;
```

Create Version (snapshot)

```
CREATE (v:Version {commit_id:$commit, node_id:$node_id, diff_summary:$summary,
created_at:datetime(), author:$user})
WITH v
MATCH (n:Node {id:$node_id})
MERGE (n)-[:HAS_VERSION]->(v)
RETURN v;
```

3. ArangoDB (AQL) — Collections & queries

Collections to create

- `nodes` (document collection)
- `edges` (edge collection)
- `versions` (document collection)
- `merge_requests` (document collection)
- `hallucinations` (document collection)

Example document for `nodes`

```
{
  "_key": "<uuid>",
  "id": "<uuid>",
  "name": "UserAuth",
  "type": "component",
  "status": "fresh",
  "vector_index_id": "vec_123",
```

```
"metadata": {},  
"created_at": "2025-08-09T...",  
"updated_at": "..."  
}
```

Create node (AQL)

```
INSERT {  
  id: @id,  
  name: @name,  
  type: @type,  
  status: 'fresh',  
  vector_index_id: @vid,  
  metadata: @meta,  
  created_at: DATE_NOW(),  
  updated_at: DATE_NOW()  
} INTO nodes  
RETURN NEW;
```

Create edge (dependency)

```
INSERT { _from: CONCAT('nodes/', @from_key), _to: CONCAT('nodes/', @to_key),  
  relation: 'depends_on', created_at: DATE_NOW() } INTO edges  
RETURN NEW;
```

Fetch dependencies (transitive up to depth)

```
LET start = DOCUMENT(CONCAT('nodes/', @node_key))  
FOR v, e, p IN 0..3 OUTBOUND start edges  
  RETURN { id: v.id, name: v.name, status: v.status, vector_index_id:  
    v.vector_index_id }
```

Mark dependents stale

```
LET updated = DOCUMENT(CONCAT('nodes/', @updated_key))  
FOR dep IN INBOUND updated edges  
  UPDATE dep WITH { status: 'stale', updated_at: DATE_NOW() } IN nodes  
RETURN dep;
```

4. Merge Request & Versioning examples (both DBs)

Create merge request (pseudo)

- Insert `merge_requests` doc: `{ merge_id, child_id, parent_id, status:'pending', text_diff, diff_summary, created_at }`

Approve merge flow

1. MergeAgent calls `VersionService.create_snapshot(parent_id)` → creates `Version` with `commit_id`.
2. Apply child content to parent (update `nodes` document or `Node` properties in Neo4j).
3. Create `Version` node and link.
4. Update merge request status to `approved`.
5. Trigger staleness propagation job (worker queue) to mark dependents.

5. Vector store & fragment metadata contract

- Each stored fragment should include: `fragment_id`, `node_id`, `text`, `commit_id`, `offset`, `created_at` and vector embedding.
- Keep pointer to vector index in node (`vector_index_id`) and store fragment metadata either in a dedicated `fragments` collection/table or in the vector store's metadata fields.
- Retrieval returns `text`, `score`, `commit_id`, `offset`.

6. Backend service contracts (method signatures)

```
# RouterService
def compose_context(node_id: str, query: str, token_budget: int) ->
ContextPackage:
    pass

# VectorService
def search(node_id: str, query: str, top_k: int) -> List[Fragment]:
    pass

# MergeService
def compute_merge(child_id: str) -> MergePreview:
    pass

# VersionService
def create_snapshot(node_id: str, author: str, reason: str) -> str: # returns
```

```
commit_id  
pass
```

7. Migration & Backups

- Regular backups of Neo4j/ArangoDB and vector indices.
 - Export versions and node content periodically to object storage (S3) for disaster recovery.
-

8. Example implementation notes & tips

- Keep `content_snapshot` out of DB for large blobs — store in object storage and save pointer in `Version` record.
 - Use `id` as UUID for cross-system references; use DB-native `_key` as needed for Arango.
 - Use transactions for critical flows (create snapshot + update parent + create merge record) to maintain consistency.
 - Avoid storing raw embeddings in main DB; keep them in vector store; store references/ids only.
-

End of LLD document.